claude-windows / 56b7f849-5d56-4770-93e0-b3b4da943700

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\56b7f849-5d56-4770-93e0-b3b4da943700\subagents\workflows\wf_49729ae7-962\agent-a74619c6d81fe8dce.jsonl`
- SHA-256: `9c7a673b7658e77f5899cdb1e3c0be1f12c56bfb2e12b6cfe1da2e4d00e0254d`
- Source modified: `2026-06-07T08:15:07+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_49729ae7-962`
- session_id: `56b7f849-5d56-4770-93e0-b3b4da943700`

Transcript

1. user (2026-06-07T08:14:15.308Z)

```
You are a tough evaluator of how DEBUGGABLE an observability report is for a README-only reader.
Scenario: indexer_no_api_key (tool: indexer)
GROUND TRUTH (what actually went wrong): The run produced 0 highlight segments because no AI-provider API key was set, so the AI segmenter silently no-op'd. It is NOT that the video has no rallies. Fix: set GEMINI_API_KEY (e.g. in a .env file) and re-run.
The report is expected to point the reader toward FAQ: "Q7".

THE FRESH READER ANSWERED:
- diagnosis: The tool ran the segmenter with AI handoff enabled but the Gemini API key was not set (GEMINI_API_KEY missing from .env). The AI provider silently exited without processing any segments, resulting in 0 confirmed segments and making it incorrectly appear as if "the video has no rallies." This is a silent failure caused by a missing credential, not a video content issue.
- fix: Create a .env file in the project root (C:\Users\avidu\Projects\badminton-highlight-indexer\) with the line GEMINI_API_KEY=your_actual_key. Obtain an API key from Google AI Studio. Verify config.json has a valid segmenter name (yolo_hybrid, gemini, or motion ? not wasb_hybrid). Re-run the processing command.
- faq_section used: README#Q7 ? How do I configure and run an AI-powered segmenter (Gemini / YOLO-hybrid)?
- could_self_debug: true
- confusing_parts: ["The segmenter name 'wasb_hybrid' is not documented and appears invalid ? the report does not explain where this came from or how to correct it to a valid option (yolo_hybrid, gemini, or motion).","The term 'silent provider no-op' is technical jargon ? clearer phrasing for users would be 'The AI provider exited without running because the API key was missing.'","The error message does not explicitly state the expected .env file path, forcing the user to refer back to Q7 for the 'project root' location."]

Also read the actual report for context:
C:\Users\avidu\AppData\Local\Temp\obseval\reports\indexer_no_api_key.report.md

Judge strictly: did the reader land on the correct ROOT CAUSE (cause_correct)? Did they find/consult the expected FAQ (faq_correct)? How large is the remaining gap (none/minor/major)? Then propose ONE concrete, specific refinement to the REPORT wording, the spec rule message, or the FAQ that would have made this scenario unambiguous to self-debug ? or "none" if it is already excellent. Don't be generous; if anything was confusing, propose a fix.
```

2. assistant (2026-06-07T08:14:21.550Z)

```
I'll analyze the report against the ground truth and the reader's answers.
```

3. user (2026-06-07T08:14:21.948Z)

```
1       # Badminton Highlight Indexer ? Video Report
2
```

```
## Run

| Field | Value |
| --- | --- |
| Tool | badminton-highlight-indexer |
| Video id | md5a (file_md5) |
| Video path | /videos/match.mp4 |
| Generated | 2026-06-07T08:10:50Z |
| Schema | 1.0 |

## Video

| Field | Value |
| --- | --- |
| Duration | 11m16s |
| Resolution | 1920x1080 |
| FPS | 59.94 fps (probed) |
| Container | mov,mp4,m4a |

## Processing stages

**validation** ? `ok` (0.0s)
  - checks_passed: 2
  - checks_total: 2

**segmentation** ? `ok` (0.0s)
  - candidate_windows: 1
  - segment_count: 0
  - total_play_s: 0 s
  - _segmenter_requested_: wasb_hybrid
  - _segmenter_actual_: wasb_hybrid
  - _config_default_: wasb_hybrid
  - _segmenter_explicitly_requested_: True
  - _matches_config_default_: True
  - _was_reingest_: False
  - _detector_: None

**ai_handoff** ? `ok` (0.0s)
  - confirmed_segments: 0
  - _ai_based_: True
  - _provider_: gemini
  - _model_: gemini-flash
  - _api_key_present_: False

## Findings & warnings

- [ERROR] **silent_provider_noop** ? 0 segments AND no AI provider API key was set ? this is almost certainly a silent provider no-op, not 'the video has no rallies'. Set the provider key (e.g. GEMINI_API_KEY in a .env file) and re-run.
  ? see **README#Q7**
  ? evidence: stages.ai_handoff.details.api_key_present=False, stages.segmentation.metrics.segment_count=0

## Verdict

**FAIL**

```

4. assistant (2026-06-07T08:14:25.695Z)

The report is clear. Let me verify the FAQ reference exists to judge whether the reader could actually consult it.

5. user (2026-06-07T08:14:27.740Z)

C:\Users\avidu\Projects\badminton-highlight-indexer\README.md:197:### Q7: How do I configure and run an AI-powered segmenter (Gemini / YOLO-hybrid)?

6. user (2026-06-07T08:14:30.409Z)

````
### Q6: How can I quickly test the indexing without waiting to parse an entire multi-GB video file?
You can pass the `--max-frames <N>` argument in CLI or `max_frames` in the POST API payload. This restricts the segmenter frame loop to process only the first $N$ sub-sampled frames, enabling you to test motion filters and court relevance checks in seconds:
```bash
python -m backend.main --video-path "C:\Users\avidu\Videos\LargeTestVideo.MP4" --max-frames 300
```
*(At 5 sub-sampled frames per second, `--max-frames 300` analyzes exactly the first 60 seconds of video).*

### Q7: How do I configure and run an AI-powered segmenter (Gemini / YOLO-hybrid)?
Both `yolo_hybrid` (default) and `gemini` need an API key for the AI provider. To use Google Gemini:
1. Create a local `.env` file in the project root (this is already in `.gitignore` to prevent secret leaks).
2. Set your Google AI Studio API key in the file:
   ```env
   GEMINI_API_KEY=your_key_here
   ```
3. Run with whichever segmenter you want:
   ```bash
   # Default: cheap candidate filtering with YOLO, then precise boundaries via Gemini
   python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter yolo_hybrid

   # Full-video Gemini analysis (more expensive, highest semantic precision per call)
   python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter gemini

   # Pure CV baseline ? no API key needed
   python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter motion
   ```
   *Or set `"default_segmenter": "yolo_hybrid"` (or `"gemini"`/`"motion"`) under `"indexing"` in `config.json`.*

The AI provider used by `yolo_hybrid` and `gemini` is controlled by `ai_provider` + `ai_model` at the top level of `config.json`. Today only `"gemini"` is registered; new providers can be added under `backend/providers/`.

### Q8: How does the `yolo_hybrid` pipeline actually save money?
Sending a multi-gigabyte video to a hosted LLM is slow and expensive. `yolo_hybrid` runs in four phases:

1. **Chunking** ? splits the video into overlapping windows (`chunk_duration`, `chunk_overlap`) so we can process incrementally.
2. **Detection + tracking** ? torchvision person detection (Faster R-CNN) + ByteTrack runs on each chunk locally (GPU-accelerated when available), measuring per-track velocity. Frames whose normalised player velocity exceeds `yolo_velocity_threshold` are kept; the rest are dropped. Adjacent active frames are merged into candidate windows, with sub-`min_action_window_duration` windows discarded.
3. **AI handoff** ? only the surviving candidate windows (padded by `ai_handoff_padding` seconds on each side) are extracted and sent to the AI provider for precise rally boundaries and shot-count estimation. These calls run in parallel up to `ai_max_workers`.
4. **DB population** ? confirmed segments are written to `play_segments` along with `shot_count`, `shot_count_confidence`, and a `detector` tag like `yolo-hybrid-gemini-2.5-flash`.

In practice this typically cuts AI-provider runtime/cost by an order of magnitude versus uploading the full video to `gemini` directly, while preserving the LLM's strength at semantic boundary detection.
````

````
### Q9: How can I debug/test the Gemini Segmenter on a small portion of a video without uploading a huge multi-GB file?
Ingesting and uploading multi-gigabyte video files to Google Cloud can be slow and expensive. We built a native **fast local trimming** developer debugging feature:
1. Open your `config.json` file.
2. Under the `"indexing"` configuration block, add a `"debug_duration"` key (in seconds):
   ```json
   "indexing": {
       "default_segmenter": "gemini",
       "debug_duration": 15.0,
       ...
   }
   ```
3. When `"debug_duration"` is set, the Gemini segmenter will instantly run a zero-re-encode, sub-second `ffmpeg` slice to create a tiny temporary clip of the first 15 seconds, upload only that tiny clip to Google AI Studio (which takes <1 second to upload and <5 seconds to process), run the model query, and clean up the temporary files automatically. This allows you to iterate on model prompts, evaluate validation overlaps, and verify rally boundaries in under 10 seconds!
### Q10: Can I trust the per-segment metadata from the `motion` segmenter?
**No.** The zero-dependency `motion` segmenter detects *timing* via pixel-difference, but its per-segment **metadata is heuristic/synthetic** ? `intensity`, `avg_speed_kmh`, the serve/smash/foul/winner **events**, and `server`/`receiver` are generated values, **not measured**. They exist so the schema is populated for demos; do not treat them as ground truth. For real shot counts and boundaries use `yolo_hybrid` (default) or `gemini`. If your observability report shows a `synthetic_motion_metadata` warning, this is why.
### Q11: I re-ran a video and my previous results changed/disappeared. Why?
Videos are keyed by the **MD5 of the file's bytes**. Re-processing the *same* file is treated as a fresh ingest: `add_video` is `INSERT OR REPLACE`, and the foreign-key cascade **deletes the previous play/candidate segments** before writing the new ones. So the earlier results are intentionally replaced, not merged. (A `reingest_replaced_prior` note in the report flags this.) Note also that re-encoding or re-downloading a video changes its bytes ? a new MD5 ? a separate record.
### Q12: What is the `*.report.json` / `*.report.md` / `*.summary.md` file next to my video?
Every processing run now writes a **per-video observability report** next to the video (or to `report.output_dir` if set in `config.json`):
````

7. assistant (2026-06-07T08:14:38.233Z)

I now have full context. Let me assess the report, the FAQ, and the reader's answer.

The report's finding (line 49) is precise and correct: it names `silent_provider_noop`, gives the exact cause ("0 segments AND no AI provider API key was set"), explicitly rules out the false interpretation ("not 'the video has no rallies'"), gives the fix ("Set the provider key (e.g. GEMINI_API_KEY in a .env file) and re-run"), points to README#Q7, and cites evidence. The reader landed exactly on the root cause and consulted Q7.

The reader's diagnosis and fix are correct and match ground truth. They flagged `wasb_hybrid` as the one genuine source of confusion. Let me check whether that confusion is legitimate, i.e., is `wasb_hybrid` actually an invalid segmenter, and does the report mislead by showing it as both requested and the config default with `_matches_config_default_: True`?

8. user (2026-06-07T08:14:41.698Z)

```
Found 6 files
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\harvest.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\gemini_refine.py
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\indexer\__init__.py
```

C:\Users\avidu\Projects\badminton-highlight-indexer\backend\indexer\wasb_hybrid.py

9. assistant (2026-06-07T08:14:50.603Z)

`wasb_hybrid` is a real registered segmenter (it has its own module `backend/indexer/wasb_hybrid.py` and is registered). So the reader's claim that "wasb_hybrid is not documented and appears invalid" is partially mistaken ? it exists in code, but the reader is right that Q7 does NOT mention it (Q7 only lists yolo_hybrid/gemini/motion). That's a real documentation gap, but it's a side-quest, not the root cause. The reader still got the root cause and fix exactly right.

The most material confusion the reader raised is the term "silent provider no-op" being jargon. That's minor ? the report immediately glosses it with "Set the provider key... and re-run," and explicitly contrasts with the false "video has no rallies" interpretation, which is the whole point.

The reader self-debugged correctly (`could_self_debug: true`), landed on the exact root cause, and consulted the exact expected FAQ (Q7). The gap is minor: the report shows `wasb_hybrid` prominently as the active segmenter without clarifying that wasb_hybrid ALSO routes through the AI provider (ai_handoff shows provider=gemini), which momentarily made the reader think the segmenter name itself was the problem. That confusion is real but didn't block self-debug.

10. user (2026-06-07T08:15:01.986Z)

Structured output provided successfully

11. assistant (2026-06-07T08:15:06.934Z)

Evaluation complete. The reader correctly identified the root cause (missing GEMINI_API_KEY causing a silent AI-provider no-op, not an empty video) and consulted the expected FAQ (README#Q7). I judged the gap as minor: the report is strong, but the segmentation stage shows `wasb_hybrid` prominently without linking it to the AI handoff, which sent the reader on a side-quest believing `wasb_hybrid` was an invalid segmenter name (it is actually a real registered segmenter at `backend/indexer/wasb_hybrid.py` that routes through the gemini provider ? it is just missing from Q7's enumeration). My one refinement folds the active-segmenter name and explicit `.env` project-root path into the finding so the reader never leaves it.